GAZİ ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BMT317 – ALGORİTMALAR

PROJE ÖDEVİ

Sıralama Algoritmalarının Performans Karşılaştırması

Öğretim Elemanı: Doç. Dr. Adem Tekerek

Öğrenci Adı Soyadı: Fuat Efe Özdemir
Öğrenci Numarası: 24181617006
Tarih: 29/10/2026

1. Giriş

Bu projenin temel amacı, Algoritmalar dersi kapsamında öğrenilen farklı sıralama algoritmalarının gerçek çalışma ortamında kodlanarak performanslarının deneysel bir çerçevede karşılaştırılmasıdır. Proje dahilinde Selection Sort, Insertion Sort, Bubble Sort, Shell Sort, Quick Sort, Merge Sort ve Heap Sort algoritmaları C++ programlama dili ile geliştirilmiştir. Hazırlanan algoritmalar; rastgele, sıralı, ters sıralı ve kısmen sıralı olmak üzere dört farklı veri dağılımında test edilmiştir. Testler n=100 ile n=100.000 arasında değişen boyutlardaki veri kümeleri üzerinde gerçekleştirilmiş olup, algoritmaların çalışma süreleri, eleman karşılaştırma sayıları, bellek içi veri hareketi (swap) sayıları ve bellek gereksinimleri analiz edilmiştir. Elde edilen deneysel veriler doğrultusunda algoritmaların güçlü/zayıf yönleri değerlendirilmiş ve teorik Big-O karmaşıklıklarının gerçek sistemlerdeki davranışlarla ne ölçüde örtüştüğü incelenmiştir.

2. Teorik Bilgi

Sıralama algoritmaları, temel olarak zaman (çalışma süresi) ve bellek (alan) karmaşıklıklarına göre sınıflandırılır. Proje kapsamında incelenen algoritmaların çalışma mantıkları ve teorik değerleri şu şekildedir:

- **Selection Sort:** Diziyi tarayarak her adımda kalan kısmın en küçük elemanını bulur ve sıradaki konuma yerleştirir. Tüm durumlarda O(n^2) zaman karmaşıklığına sahiptir.

- **Insertion Sort:** Dizideki her elemanı sırayla alır ve kendinden önceki sıralanmış kısmın içinde uygun yere yerleştirir. Ortalama ve en kötü durumda O(n^2) zaman karmaşıklığına sahiptir ancak sıralı veri setlerinde performansı O(n) seviyesine yaklaşır.

- **Bubble Sort:** Yan yana olan iki elemanı karşılaştırıp gerekiyorsa yer değiştirerek en büyük elemanı dizinin sonuna iter. Ortalama ve en kötü durumda O(n^2) zaman karmaşıklığı ile çalışır.

- **Shell Sort:** Insertion Sort algoritmasının geliştirilmiş halidir; ardışık olmayan, belirli bir aralık (gap) mesafesindeki elemanları karşılaştırarak diziyi kabaca sıralar ve aralığı yavaşça 1'e düşürür. Zaman karmaşıklığı doğrudan kullanılan aralık dizisine bağlı olarak değişmektedir.

- **Quick Sort:** Bir pivot eleman seçer, diziyi pivottan küçükler ve büyükler olarak iki alt diziye böler. Ortalama durumda O(n \log n) ile oldukça hızlı çalışırken, pivot seçimine bağlı olarak en kötü durumda O(n^2) zaman karmaşıklığına düşebilir.

- **Merge Sort:** Diziyi tek eleman kalana kadar sürekli ikiye böler, ardından bu parçaları sıralı bir bütün oluşturacak şekilde birleştirir. Tüm durumlarda O(n \log n) zaman karmaşıklığına sahiptir.

- **Heap Sort:** Veriden bir maksimum yığın ağaç yapısı oluşturur, en büyük elemanı sona atar ve kalan elemanlar için yığını yeniden düzenler. Tüm durumlarda O(n \log n) sürede çalışır.

3. Algoritmaların Gerçeklenmesi

Proje kapsamındaki tüm algoritmalar, C++ standart kütüphanesindeki hazır sıralama fonksiyonları kullanılmadan sıfırdan kodlanmıştır. Geliştirme aşamasında alınan tasarım kararları ve yöntemler aşağıda detaylandırılmıştır:

- **Performans Sayaçları:** Algoritmaların performansını ölçmek için koda bir Metrics veri yapısı eklenmiştir.

  Karşılaştırma Sayacı, dizideki iki elemanın büyüklük/küçüklük durumunun kontrol edildiği her koşulda artırılmıştır.

  Veri Hareketi Sayacı ise dizideki elemanların karşılıklı yer değiştirdiği veya Insertion Sort/Merge Sort gibi algoritmalarda bir elemanın başka bir konuma kopyalandığı her adımda sayılarak maliyeti hesaplanmıştır.

- **Doğruluk Kontrolü:** Performans ölçümlerine geçilmeden önce, kodlanan her algoritmanın diziyi gerçekten küçükten büyüğe sıralayıp sıralamadığı özel olarak yazılan isSorted(array) fonksiyonu ile her döngüde kontrol edilmiştir.

- **Quick Sort Pivot Seçimi:** Standart algoritmada pivot olarak orta eleman kullanılmıştır. Sıralı ve ters sıralı büyük veri kümelerinde ilk veya son elemanın pivot seçilmesi algoritmayı en kötü duruma iterek özyineleme sınırını aşmasına ve stack overflow hatasına neden olduğundan bu yöntem tercih edilmiştir.

  Yönergedeki bonus çalışması kapsamında ilk elemanlı pivotlu versiyon (Quick_FirstPivot) da koda dahil edilip bu zayıflık deneysel olarak kanıtlanmıştır.

- **Shell Sort Gap (Aralık) Seçimi:** Temel algoritmada aralık değeri dizinin yarısından başlayıp sürekli yarıya inerek (n/2, n/4, … , 1) ilerleyen standart dizi kullanılmıştır.

  Bonus çalışması kapsamında ise asimetrik 3x+1 (1, 4, 13, 40...) gap serisi kullanılarak Shell_Knuth algoritması oluşturulmuş ve boşluk dizisinin performansa etkisi test edilmiştir.

4. Deneysel Çalışma

Deneyler, sonuçların tutarlılığını ve güvenilirliğini sağlamak amacıyla aynı ortamda gerçekleştirilmiştir. Deneysel çalışma tasarımında, önceki incelemede belirtilen eksiklikler de giderilerek aşağıdaki esaslar dikkate alınmıştır:

- **Veri Büyüklükleri:** Algoritmalar sırasıyla n = 100, 1.000, 5.000, 10.000, 50.000$ ve 100.000 elemanlı diziler üzerinde test edilmiştir. Zaman karmaşıklığı O(n^2)$olan Selection, Insertion ve Bubble Sort algoritmaları ile ilk elemanı

pivot alan Quick 10.000'den büyük boyutlarda aşırı uzun süreceği için, bu boyutlarda test edilememiş ve tabloya "Limit Aşımı" olarak kaydedilmiştir.

- **Veri Dağılımları:** Her bir algoritma; Rastgele, Sıralı (küçükten büyüğe), Ters Sıralı (büyükten küçüğe) ve Kısmen Sıralı (büyük bölümü sıralı, %10'u rastgele değiştirilmiş) olmak üzere 4 farklı veri durumu üzerinde çalıştırılmıştır.
- **Adil Karşılaştırma ve Tekrarlanabilirlik:** Her algoritma için her zaman aynı başlangıç dizisinin kopyası kullanılmıştır. Arka plandaki işletim sistemi süreçlerinden veya ilk çalıştırma maliyetlerinden etkilenmemek için her deney 5 kez tekrarlanmış ve ölçümlerin aritmetik ortalaması alınarak nihai sonuca ulaşılmıştır.
- **Performans Kriterleri (Ölçüm Kapsamı):** Süre ölçümleri C++ std::chrono::high_resolution_clock kullanılarak milisaniye cinsinden yapılmıştır. Zaman ölçümü yalnızca sıralama algoritmalarının çağrıldığı satırları kapsamış, veri üretimi veya dizilerin kopyalanması gibi yardımcı işlemler ölçüme dahil edilmemiştir. Ayrıca her algoritmaya sayaçlar eklenerek eleman karşılaştırma ve yer değiştirme (swap/hareket) sayıları kesin olarak ölçülmüştür.
- **Donanım ve Yazılım Ortamı:** Testler, 11. Nesil Intel Core i9-11900F (2.50 GHz) işlemci, 32 GB RAM ve Windows 11 işletim sistemine sahip bir bilgisayarda gerçekleştirilmiştir.

  Proje, C++20 standardıyla CLion 2024.2.2 IDE'si üzerinde kodlanmış ve MinGW (GCC) derleyicisi ile derlenmiştir.

5. Performans Sonuçları

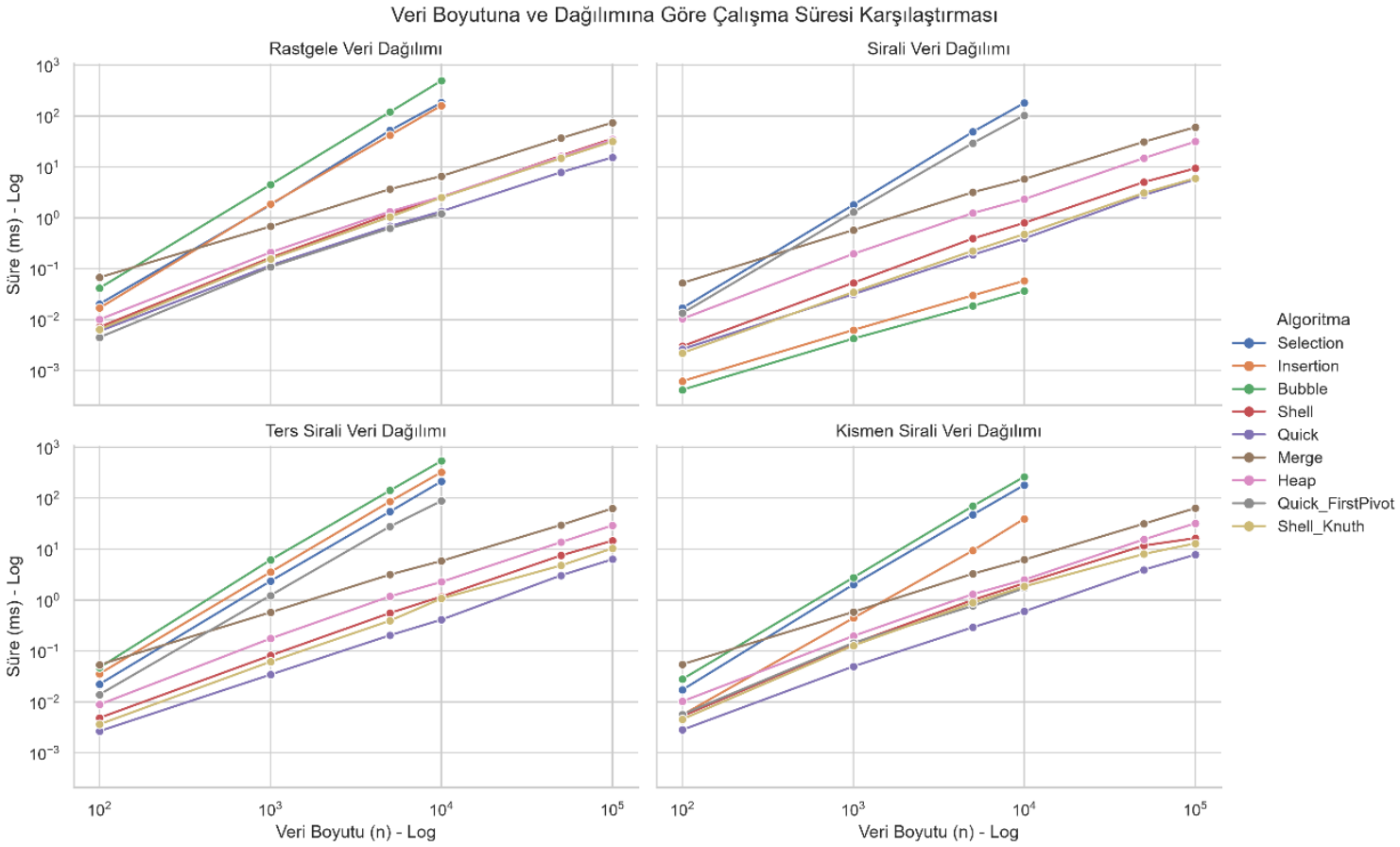
Elde edilen tüm ham veriler; boyut, veri tipi, ortalama süre, karşılaştırma ve swap sayılarını içerecek şekilde deney_sonuclari.csv dosyasına kaydedilmiştir. Grafiksel analize geçilmeden önce, algoritmaların genel eğilimlerini sayısal olarak özetlemek amacıyla rastgele veri dağılımındaki temel boyutlara ait ölçümler Tablo 1'de sunulmuştur.

Tablo 1: Rastgele Veri Dağılımında Temel Boyutlar İçin Ortalama Performans Özeti

Algoritma	***Süre (n=1.000)***	***Süre (n=100.000)***	***Karşılaştırma (n=100.000)***	***Swap (n=100.000)***
Selection Sort	*1.78 ms*	*Limit Aşımı*	-	-
Insertion Sort	*1.84 ms*	*Limit Aşımı*	-	-
Bubble Sort	*4.48 ms*	*Limit Aşımı*	-	-
Quick Sort	*0.11 ms*	*15.31 ms*	*2.114.410*	*438.400*
Shell Sort	*0.16 ms*	*35.95 ms*	*4.198.588*	*4.249.059*
Merge Sort	*0.67 ms*	*73.92 ms*	*1.536.473*	*3.337.856*
Heap Sort	*0.20 ms*	*34.45 ms*	*3.019.691*	*1.575.048*

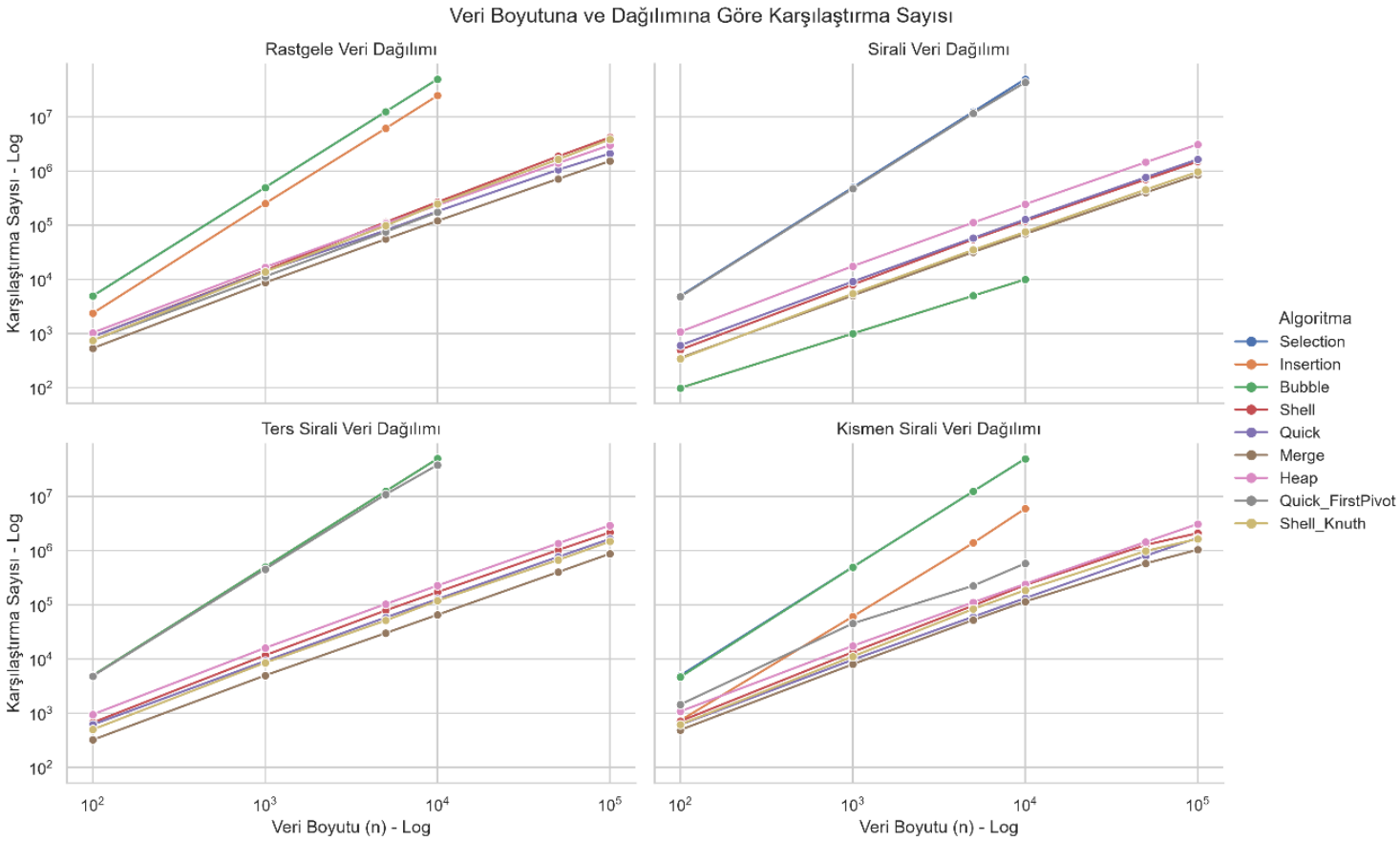
Not: Zaman karmaşıklığı O(n^2) olan algoritmalar büyük veri kümelerinde süreyi aşırı uzattığı için yönerge gereği test dışı bırakılmış ve tablolara "Limit Aşımı" olarak kaydedilmiştir.

5.1. Veri Boyutuna Göre Çalışma Süresi Karşılaştırması



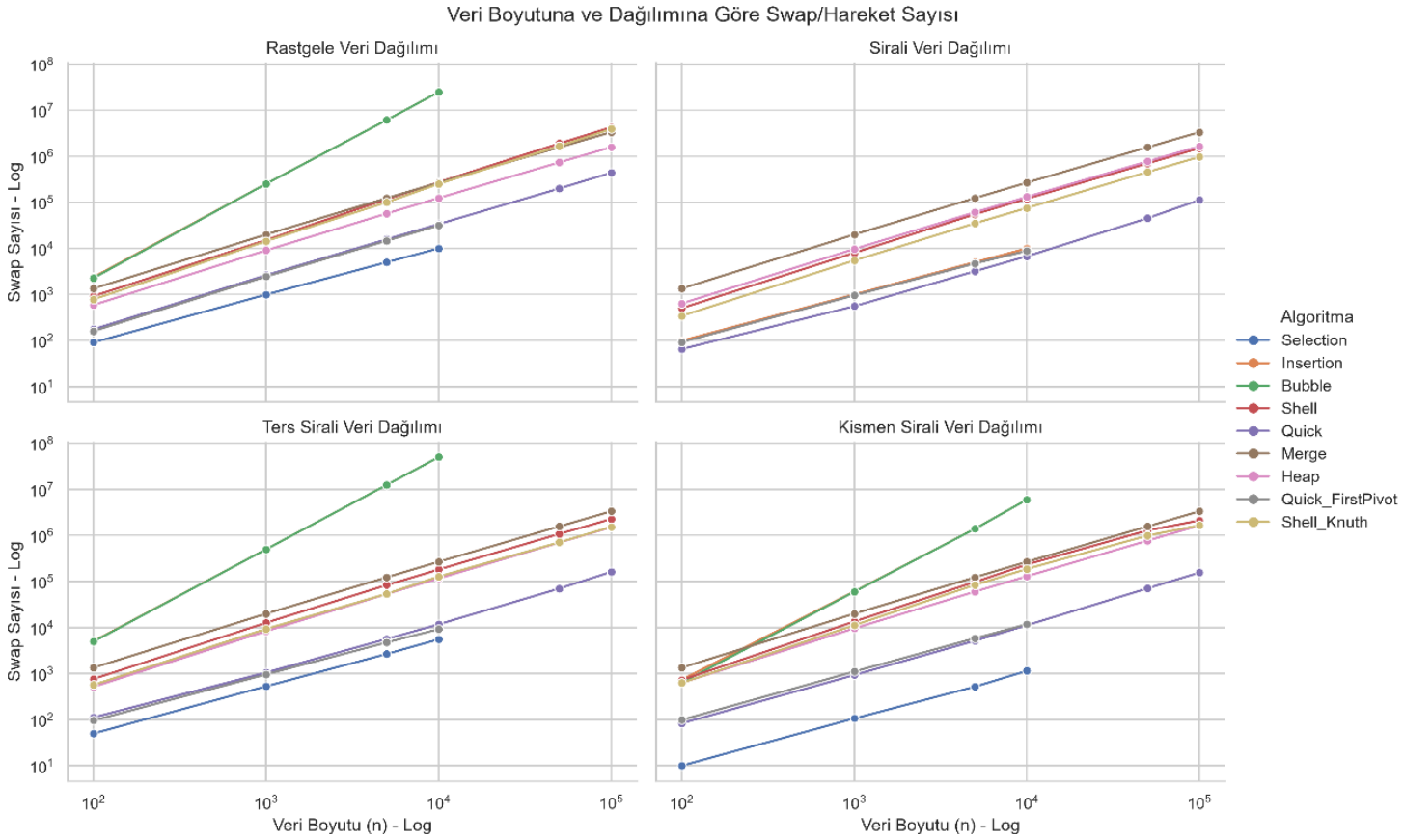
Grafik 1'de, algoritmaların çalışma sürelerinin veri boyutuna göre değişimi logaritmik eksende sunulmuştur. O(n^2) karmaşıklığına sahip Selection, Insertion ve Bubble Sort algoritmalarının rastgele ve ters sıralı verilerde beklendiği gibi büyüme eğilimini sergilediği görülmektedir. Ancak sıralı veri dağılımında Insertion ve Bubble Sort'un sürelerinin logaritmik eksenin en altına inerek O(n) (best-case) performansına ulaştığı fark edilmektedir. Quick_FirstPivot algoritması rastgele veride oldukça hızlıyken, sıralı ve ters sıralı verilerde karesel büyüme çizgisine çıkarak zayıflığını göstermiştir. Merge, Heap, Quick ve Shell Sort ise tüm senaryolarda daha yatay bir büyüme sergileyerek büyük veri setlerindeki zaman avantajlarını kanıtlamıştır.

5.2 Veri Boyutuna Göre Karşılaştırma Sayısı



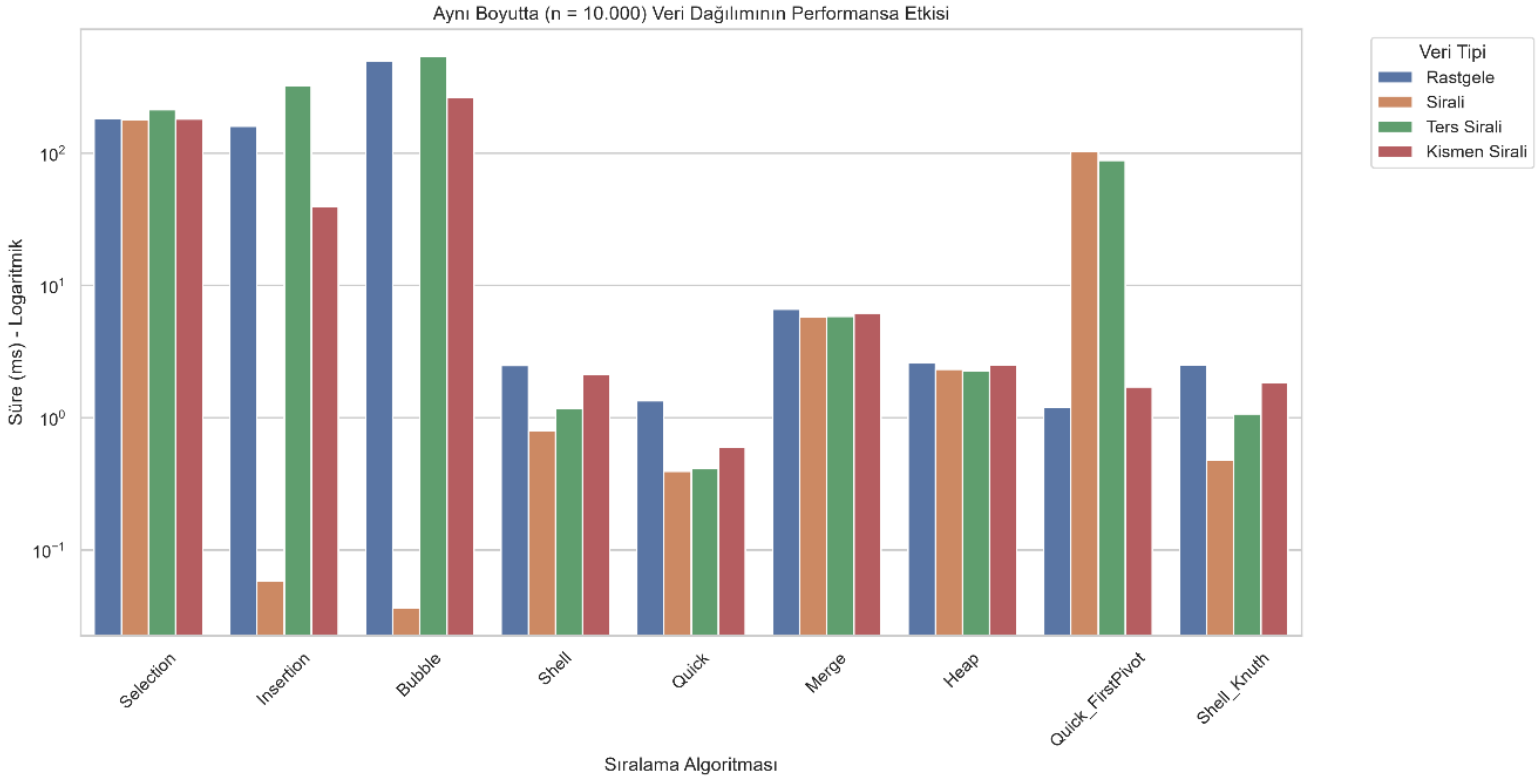
Grafik 2'de algoritmaların elemanları birbirleriyle kıyaslama sayıları gösterilmektedir. Selection Sort'un, verinin dizilimine (Rastgele, Sıralı, Ters Sıralı, Kısmen Sıralı) bakılmaksızın her durumda aynı yüksek karşılaştırma sayısına sahip olduğu tespit edilmiştir. Bubble ve Insertion Sort, sıralı veride erken durma optimizasyonları sayesinde minimum karşılaştırma yaparken, ters sıralı veride tepe noktasına ulaşmıştır. Quick_FirstPivot'un sıralı ve ters sıralı verilerde karşılaştırma sayısının O(n^2) algoritmalarıyla aynı seviyelere çıktığı, standart Quick, Merge ve Heap Sort'un ise logaritmik bölmeler sayesinde tüm durumlarda karşılaştırma sayılarını optimize ettiği görülmektedir.

5.3 Veri Boyutuna Göre Swap / Veri Hareketi Sayısı



Grafik 3, algoritmaların bellek içi atama veya yer değiştirme (swap) maliyetlerini yansıtmaktadır. En dikkat çekici tespit, zaman ve karşılaştırma açısından yavaş olan Selection Sort'un, rastgele, ters sıralı ve kısmen sıralı verilerde en düşük swap sayısına sahip algoritma olmasıdır. Bu durum, algoritmanın yer değiştirme işlemini sadece dış döngüde yapmasından kaynaklanmaktadır. Bubble ve Insertion Sort, ters sıralı verilerde maksimum veri hareketine neden olarak belleği en çok yoran algoritmalar olmuştur. Merge Sort'un ise veri dağılımından bağımsız olarak her dört grafikte de swap sayılarının birebir aynı eğride kaldığı, yapısının verinin diziliminden etkilenmediği doğrulanmıştır.

5.4 Aynı Boyutta Veri Dağılımının Performansa Etkisi (n = 10.000)



Grafik 4'te, n=10.000 boyutundaki sabit veri setinde algoritmaların başlangıç veri dağılımlarına verdiği tepkiler grafikle özetlenmiştir. Insertion ve Bubble Sort'un rastgele ve ters sıralı verilerde sürelerinin yüksek olduğu, ancak sıralı veride 1 ms'nin oldukça altına inerek devasa bir performans yükselmesi yaşandığı görülmektedir. Quick_FirstPivot algoritması rastgele ve kısmen sıralı verilerde standart Quick Sort ile benzer sonuçlar verirken, sıralı ve ters sıralı verilerde karesel süre bandına çıkarak adeta bir Bubble Sort gibi davranmıştır. Merge ve Heap Sort çubuklarının ise her dört veri tipinde de kendi içlerinde neredeyse aynı hizada kalması, bu algoritmaların veri bağımsız ve kararlı performanslarını kanıtlamaktadır.

6. Karşılaştırma ve Tartışma

Elde edilen deneysel veriler doğrultusunda, proje yönergesinde belirtilen temel araştırma sorularının analizleri aşağıda sunulmuştur:

- **O(n^2) algoritmaların çalışma süresi veri boyutu arttıkça nasıl değişmiştir?**
  O(n^2) karmaşıklığına sahip Selection, Insertion ve Bubble Sort algoritmalarının çalışma süreleri, veri boyutu arttıkça büyümüştür. Örneğin rastgele veri kümesinde Selection Sort n=1.000 için 1.7 ms sürerken, veri boyutu n=10.000 olduğunda süre yaklaşık 183 ms seviyesine çıkmıştır. Bu durum veri miktarı 10 kat arttığında işlem yükünün 10^2 = 100 kat artması teorik beklentiyle ve karesel büyümeyle uyumludur.

- **O(n \log n)$algoritmalar neden büyük veri kümelerinde avantajlıdır?**
  Merge Sort, Quick Sort ve Heap Sort gibi algoritmalar, veri kümesini alt parçalara bölerek işler. n=100.000 boyutundaki rastgele veride O(n^2) algoritmalar sistemin limitlerini aşarken, O(n \log n) algoritmalar işlemi oldukça kısa sürelerde tamamlamıştır. log n faktörü veri boyutu büyüdükçe çok yavaş arttığı için, bu algoritmalar büyük n değerlerinde karesel algoritmalara kıyasla doğrusal büyümeye çok daha yakın bir artış göstererek büyük veri setlerinde avantaj sağlar.

- **Insertion Sort sıralı verilerde neden başarılı olabilmektedir?**
  Deney sonuçlarına göre n=10.000 boyutundaki sıralı veride Insertion Sort sadece 0.05 ms sürmüş ve 9.999 adet (yani n-1) karşılaştırma yapmıştır. Bu başarının temel nedeni, uygulamanın gerçekten sıralı durumda erken çıkış yapacak şekilde tasarlanmış olmasıdır. Dizideki elemanlar zaten sıralı olduğunda, algoritmanın iç döngüsü ilk karşılaştırmada elemanın yerinin doğru olduğunu tespit edip çıkış yapar. Böylece iç içe döngüler çalışmaz ve algoritmanın performansı O(n) seviyesine ulaşır.

- **Quick Sort'un performansı pivot seçimine neden bağlıdır?**
  n=10.000 boyutundaki sıralı veride orta elemanı pivot alan standart Quick Sort 0.39 ms sürerken, ilk elemanı pivot alan Quick_FirstPivot 103.2 ms sürmüş ve n=50.000'de yığın taşmasına neden olarak testi geçememiştir. Dizi zaten sıralıyken ilk veya son elemanı pivot seçmek, dizinin her adımda bir yanda 1, diğer yanda n-1 eleman kalacak şekilde dengesiz bölünmesine yol açar. Bu da özyineleme derinliğini artırarak algoritmayı en kötü durum senaryosu olan O(n^2) karmaşıklığına düşürür.

- **Merge Sort'un performansı farklı veri dağılımlarında neden daha kararlı olabilir?**
  Tablodaki verilere bakıldığında Merge Sort'un rastgele, sıralı, ters sıralı ve kısmen sıralı durumlarında n=100.000 için swap sayısı tüm durumlarda tam olarak 3.337.856'dır. Aynı hareket sayısı, uygulamanın yapısal olarak her veri dağılımında benzer işlemleri gerçekleştirdiğini desteklemektedir. Sürelerin de birbirine yakın değerlerde elde edilmesi ayrı bir tespit olarak performansın veri dağılımından bağımsız ve kararlı olduğunu göstermektedir. Bu nedenle tüm

durumlardaki zaman karmaşıklığı O(n \log n)'dir.

- **Heap Sort ile Merge Sort'un büyük veri kümelerindeki performansı nasıl karşılaştırılmaktadır?**
  n=100.000 boyutundaki rastgele veride Heap Sort yaklaşık 34.4 ms sürerken, Merge Sort 73.9 ms sürmüştür. Teorik olarak her iki algoritma da O(n \log n) olmasına rağmen ölçümler Heap Sort'un daha hızlı sonuç verdiğini göstermektedir. Heap Sort yerinde sıralama yaparken, Merge Sort yardımcı bir diziye ihtiyaç duyar. Bu hız farkı bellek tahsisi, ek kopyalama maliyetleri ve kullanılan derleyici optimizasyonlarının önbellek üzerindeki etkileriyle açıklanabilir.

- **Shell Sort'un performansı gap dizisinden nasıl etkilenmektedir?**
  n=100.000 rastgele veride klasik n/2 gap dizisini kullanan Shell Sort 35.9 ms sürmüşken, 3x+1 formülünü kullanan Shell_Knuth 31.6 ms sürmüş ve karşılaştırma sayıları azalmıştır. Deney sonuçları, Knuth dizisinin daha az karşılaştırma ve daha kısa süre ürettiğini göstermektedir. Bunun açıklaması, 3x+1 formülünün, n/2 dizisinde görülebilen aynı elemanları gereksiz yere tekrar kıyaslama riskini azaltmasıdır.

- **Teorik Big-O değerleri deneysel sonuçlarla örtüşmekte midir?**
  Elde edilen istatistikler, teorik büyüme eğilimleriyle büyük oranda uyumludur. O(n^2) sınıfındaki algoritmaların veri boyutu arttıkça yükselen süreleri ile O(n \log n) sınıfındaki algoritmaların büyük verideki üstünlüğü gözlemlenmiştir. Aynı zamanda, kod optimizasyonları sayesinde Insertion ve Bubble Sort'un sıralı verideki O(n) davranışı ile dengesiz pivot seçimi sonucunda Quick Sort'un O(n^2) davranışı gibi kavramların pratiğe yansıdığı deneysel olarak desteklenmiştir.

7. Sonuç

Bu proje ile temel sıralama algoritmaları somut bir biçimde kodlanarak gerçek çalışma davranışları deneysel olarak analiz edilmiştir. Gerçekleştirilen testler sonucunda, algoritmaların birbirlerine karşı mutlak bir üstünlüğü bulunmadığı; performansın doğrudan veri büyüklüğüne, verinin başlangıç dağılımına ve sistem kısıtlarına bağlı olduğu görülmüştür.

Deney bulgularına dayanarak algoritmaların karakteristik özellikleri şu şekilde özetlenebilir:

- **Selection ve Bubble Sort:** Büyük veri kümelerinde karesel büyüme eğilimleri nedeniyle oldukça verimsiz kalmışlardır. Ancak Bubble Sort'un erken durdurma optimizasyonu, sıralı verilerde ciddi bir performans artışı sağlamıştır.

- **Insertion Sort:** Rastgele ve ters sıralı büyük verilerde zayıf kalsa da, erken çıkış yapabilme yeteneği sayesinde küçük boyutlu veya büyük ölçüde sıralı dizilerde son derece avantajlı ve maliyetsiz bir çözüm olabileceğini kanıtlamıştır.

- **Quick Sort:** Bu uygulamada, rastgele verilerde en hızlı sonuçları veren

algoritmalardan biri olmuştur. Ancak başarısının seçilen pivot yöntemine ne kadar bağımlı olduğu, ilk eleman pivotlu varyasyonun sıralı verilerde çökelmesiyle (limit aşımı) açıkça görülmüştür.

- **Merge Sort:** Farklı veri dağılımlarında O(n \log n) zaman karmaşıklığını koruması, süre ve hareket (swap) sayılarının görece kararlı olması bakımından öne çıkmıştır.

- **Heap Sort:** Merge Sort gibi O(n \log n) zaman karmaşıklığı sunarken, yerinde sıralama yapması sayesinde ek bellek gereksinimi açısından güçlü bir alternatif olduğunu göstermiştir.

8. Kaynakça

[1] Çölkesen, R. (2002). *Veri Yapıları ve Algoritmalar*. Papatya Yayıncılık.
[2] Weiss, M. A. (2013). *Data Structures and Algorithm Analysis in C++* (4. Baskı). Pearson.
[3] Şeker, S. E. (2007). *Sıralama Algoritmaları*. Bilgisayar Kavramları. [Çevrimiçi].
[4] GeeksforGeeks (2024). *Analysis of Different Sorting Algorithms*. [Çevrimiçi].